

Data Analysis with Python

Week 3: Intro to Data Wrangling

Justinas Grigaitis, 2026-10-01

Outline for next weeks

Data Analysis with Python

- Weeks 1-2: python fundamentals
- ***Week 3: fundamentals of data wrangling***
- Week 4: data reshaping and analysis
- Week 5: data visualisation, geospatial analysis, and econometrics
 - + Quiz (20% of grade)
- Week 6: sample project, agentic engineering, and student idea pitches

Plan for today

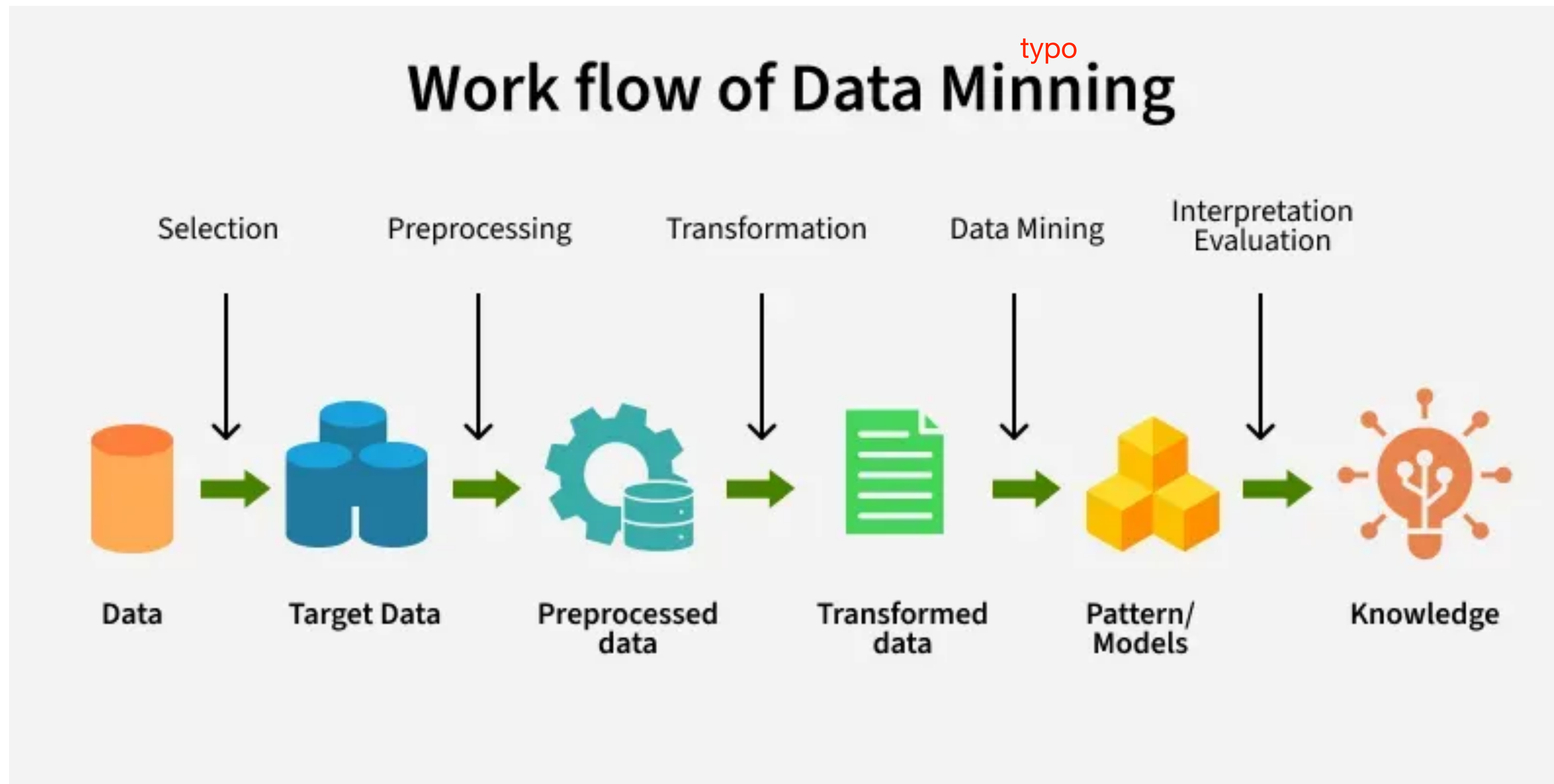
Fundamentals of data analysis

- Feedback (~10 min)
- Intro to data wrangling (~20 min)
- Activity - project ideas (~15 min)
- Coding - intro to data analysis (~45 min)

Intro to data wrangling

Data analysis

Data cleaning -> wrangling -> mining -> interpreting



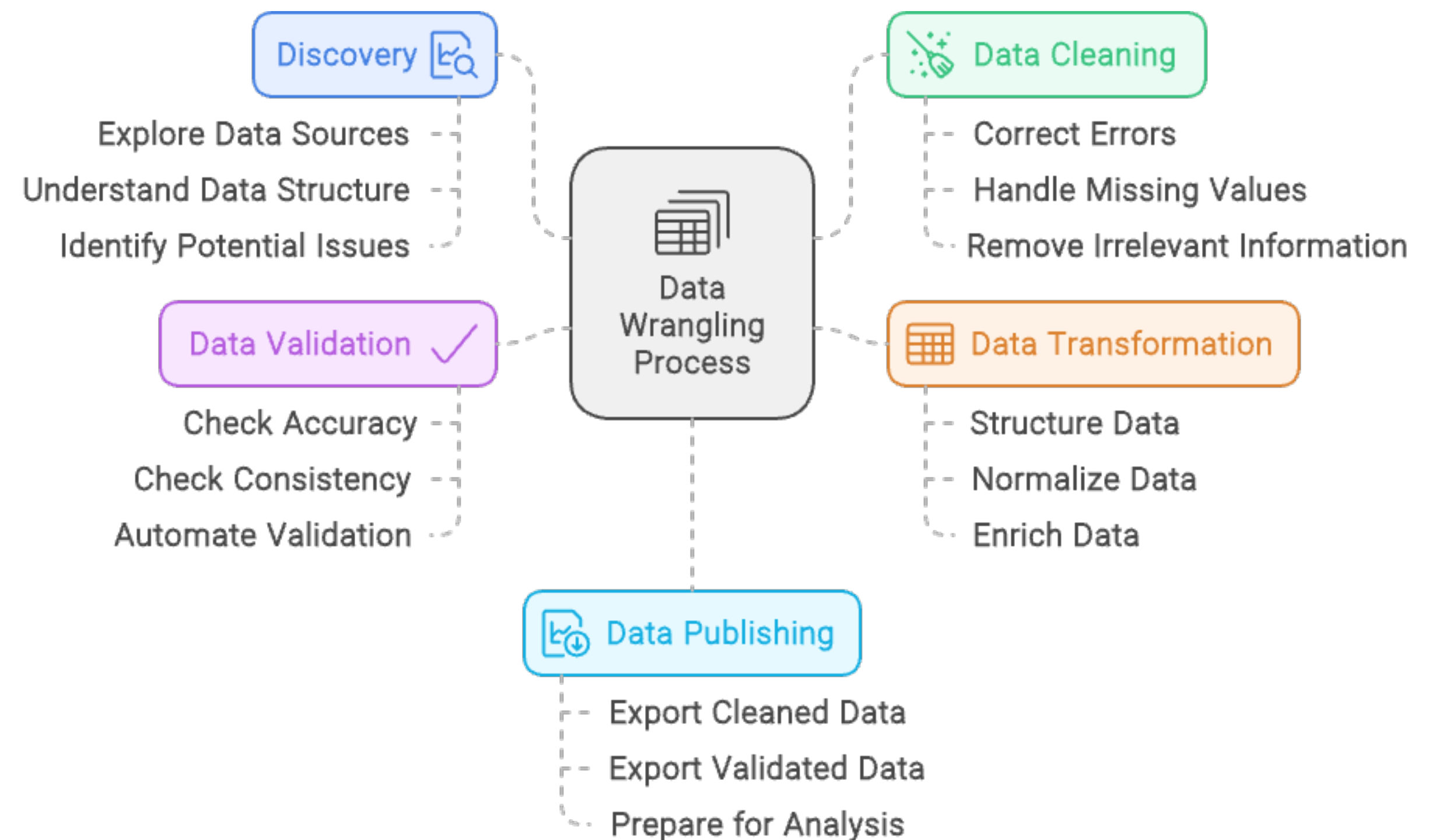
Source:

<https://www.geeksforgeeks.org/blogs/what-is-data-mining-a-complete-beginners-guide/>

Crash course on data wrangling

Plan -> clean -> validate -> output

- Discovery - diagnostics and plan
- Structuring - combine and reshape
- Cleaning - edit variables
- Enriching - add new variables
- Validating - sanity checks
- Publishing - output
- And then analyse!



Sources:

<https://www.alteryx.com/glossary/data-wrangling>

<https://www.datacamp.com/blog/what-is-data-wrangling>

Tidy data

Happy datasets are all alike...

Source: <https://vita.had.co.nz/papers/tidy-data.pdf>

1. Each variable forms a column.
2. Each observation forms a row.
3. Each type of observational unit forms a table.

country	year	cases	population
Afghanistan	1999	745	19987071
Afghanistan	2000	2666	20595360
Brazil	1999	37737	172006362
Brazil	2000	80488	174504898
China	1999	212258	1272915272
China	2000	216766	1280428583

variables

country	year	cases	population
Afghanistan	1999	745	19987071
Afghanistan	2000	2666	20595360
Brazil	1999	37737	172006362
Brazil	2000	80488	174504898
China	1999	212258	1272915272
China	2000	216766	1280428583

observations

country	year	cases	population
Afghanistan	999	745	19987071
Afghanistan	000	2666	20595360
Brazil	999	37737	172006362
Brazil	000	80488	174504898
China	999	212258	1272915272
China	000	216766	1280428583

values

Sad data

country	year	m014	m1524	m2534	m3544	m4554	m5564	m65	mu	f014
AD	2000	0	0	1	0	0	0	0	—	—
AE	2000	2	4	4	6	5	12	10	—	3
AF	2000	52	228	183	149	129	94	80	—	93
AG	2000	0	0	0	0	0	0	1	—	1
AL	2000	2	19	21	14	24	19	16	—	3
AM	2000	2	152	130	131	63	26	21	—	1
AN	2000	0	0	1	2	0	0	0	—	0
AO	2000	186	999	1003	912	482	312	194	—	247
AR	2000	97	278	594	402	419	368	330	—	121
AS	2000	—	—	—	—	1	1	—	—	—

Table 9: Original TB dataset. Corresponding to each ‘m’ column for males, there is also an ‘f’ column for females, f1524, f2534 and so on. These are not shown to conserve space. Note the mixture of 0s and missing values (—). This is due to the data collection process and the distinction is important for this dataset.

Happy data

country	year	sex	age	cases
AD	2000	m	0-14	0
AD	2000	m	15-24	0
AD	2000	m	25-34	1
AD	2000	m	35-44	0
AD	2000	m	45-54	0
AD	2000	m	55-64	0
AD	2000	m	65+	0
AE	2000	m	0-14	2
AE	2000	m	15-24	4
AE	2000	m	25-34	4
AE	2000	m	35-44	6
AE	2000	m	45-54	5
AE	2000	m	55-64	12
AE	2000	m	65+	10
AE	2000	f	0-14	3

(b) Tidy data

Tidy tools in pandas

Cheat Sheet is all you need

Data Wrangling with pandas Cheat Sheet

<http://pandas.pydata.org>

Tidy Data – A foundation for wrangling in pandas

In a tidy data set:

Each **variable** is saved in its own **column** & Each **observation** is saved in its own **row**

Tidy data complements pandas's **vectorized operations**. pandas will automatically preserve observations as you manipulate variables. No other format works as intuitively with pandas.

$M * A = F$

Syntax – Creating DataFrames

	a	b	c
1	4	7	10
2	5	8	11
3	6	9	12

```
df = pd.DataFrame(  
    {"a" : [4 ,5, 6],  
     "b" : [7, 8, 9],  
     "c" : [10, 11, 12]},  
    index = [1, 2, 3])  
Specify values for each column.
```

```
df = pd.DataFrame(  
    [[4, 7, 10],  
     [5, 8, 11],  
     [6, 9, 12]]
```

Reshaping Data – Change the layout of a data set

`pd.melt(df)`
Gather columns into rows.

`df.pivot(columns='var', values='val')`
Spread rows into columns.

`df.sort_values('mpg')`
Order rows by values of a column (low to high).

`df.sort_values('mpg', ascending=False)`
Order rows by values of a column (high to low).

`df.rename(columns = {'y': 'year'})`
Rename the columns of a DataFrame

`df.sort_index()`
Sort the index of a DataFrame

`df.reset_index()`
Reset index of DataFrame to row numbers, moving index to columns.

<https://pandas.pydata.org/Pandas Cheat Sheet.pdf>
https://pandas.pydata.org/docs/user_guide/index.html

Sad data

country	year	m014	m1524	m2534	m3544	m4554	m5564	m65	mu	f014
AD	2000	0	0	1	0	0	0	0	—	—
AE	2000	2	4	4	6	5	12	10	—	3
AF	2000	52	228	183	149	129	94	80	—	93
AG	2000	0	0	0	0	0	0	1	—	1
AL	2000	2	19	21	14	24	19	16	—	3
AM	2000	2	152	130	131	63	26	21	—	1
AN	2000	0	0	1	2	0	0	0	—	0
AO	2000	186	999	1003	912	482	312	194	—	247
AR	2000	97	278	594	402	419	368	330	—	121
AS	2000	—	—	—	—	1	1	—	—	—

Table 9: Original TB dataset. Corresponding to each ‘m’ column for males, there is also an ‘f’ column for females, f1524, f2534 and so on. These are not shown to conserve space. Note the mixture of 0s and missing values (—). This is due to the data collection process and the distinction is important for this dataset.

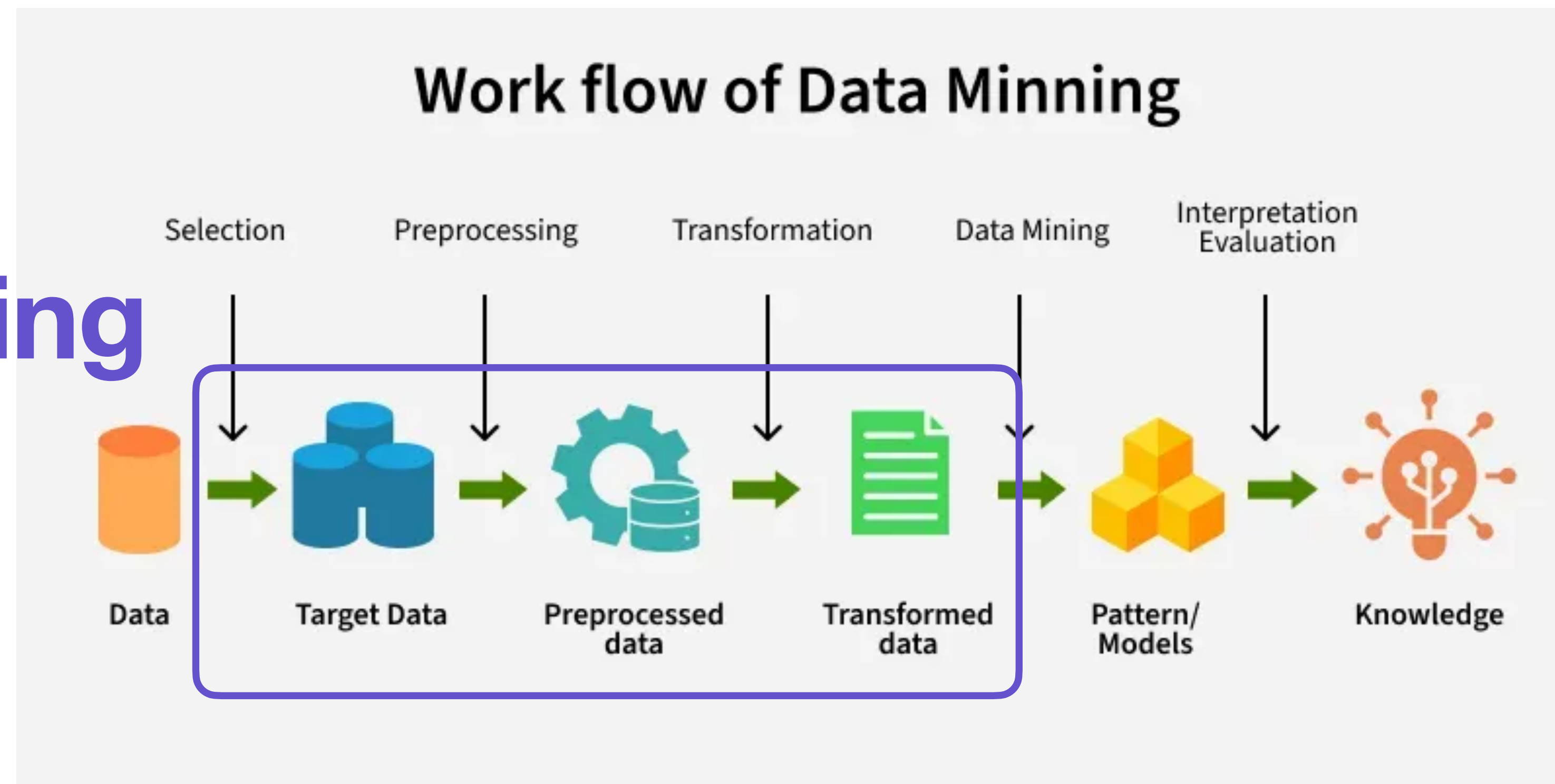
Happy data

country	year	sex	age	cases
AD	2000	m	0-14	0
AD	2000	m	15-24	0
AD	2000	m	25-34	1
AD	2000	m	35-44	0
AD	2000	m	45-54	0
AD	2000	m	55-64	0
AD	2000	m	65+	0
AE	2000	m	0-14	2
AE	2000	m	15-24	4
AE	2000	m	25-34	4
AE	2000	m	35-44	6
AE	2000	m	45-54	5
AE	2000	m	55-64	12
AE	2000	m	65+	10
AE	2000	f	0-14	3

(b) Tidy data

How do we get there?

Wrangling



Selecting

- Remove
- Variables/rows

Cleaning

- Edit/add
- Variables

Merging

- Append
- Join

Reshaping

- Aggregate
- Melt and Pivot

Preparation phase

Investigate, plan, and prepare

- Plan and pseudocode: input/output
- Understand your (meta)data:
 - `df.describe()` and `df.info()`
 - Count missing and unique values, min/max/mean
 - Basic plots (histograms if single, bars if groups, lines if time series)
- Harmonize and focus: `df.rename(columns=)` and `df.drop(columns=)`

Filtering phase - row operations

Indexing and selecting data

- **Filtering:** always based on condition, never on position
 - `df.loc[], np.where(), np.select()`
- **Missing values:** `df.dropna(), df.fillna()`
- **Duplicates:** `df.drop_duplicates()` (or `df.groupby(), df.agg()`)
- User guide

Cleaning phase - column operations

Working with numeric, text, and datetime data

Convert to appropriate variable type, then:

- **Numeric:** basic math
- **String:** slice, replace, extract parts (regex), split, dummy variables
 - Reference, User guide
- **Date:** extract hours, days, weeks, months, years
 - Reference, User guide
- **Bonus:** categorical data, base python objects (dicts, lists)

Import/Export

Making use of base Python

- Choose the right pandas function based on file format (.csv, .xlsx, .dta, etc.)
 - Bonus: specify variable types upon importing (e.g., if you know which variables are date/numeric)
 - Careful not to lose any data upon import!
- Use *os* or *pathlib* libraries to read file names and create folders
- Use loops, dictionaries, lists when importing and exporting for automation
- User guide

Python for frontend

Web apps with Python

Easy way to the outside world

- Python is primarily a back-end oriented language
- New libraries adapted it for front-end: Django, Flask, FastAPI, Plotly Dash
- More recently, a wave of very user-friendly tools: Streamlit, Gradio, Taipy, etc.
- Most importantly, a change in mindset in the age of GenAI
- Tutorials, videos: [fast prototyping](#), [data reports vs apps](#), <https://30days-tmp.streamlit.app/>

```
import streamlit as st
import pandas as pd
import numpy as np

st.write("Streamlit supports a wide range of data visualizations, i

all_users = ["Alice", "Bob", "Charly"]
with st.container(border=True):
    users = st.multiselect("Users", all_users, default=all_users)
    rolling_average = st.toggle("Rolling average")

np.random.seed(42)
data = pd.DataFrame(np.random.randn(20, len(users)), columns=users)
if rolling_average:
    data = data.rolling(7).mean().dropna()

tab1, tab2 = st.tabs(["Chart", "Dataframe"])
tab1.line_chart(data, height=250)
tab2.dataframe(data, height=250, use_container_width=True)
```

Share

Streamlit supports a wide range of data visualizations, including [Plotly](#), [Altair](#), and [Bokeh charts](#). 🇮🇹 And with over 20 input widgets, you can easily make your data interactive!

Users

Alice ×

Bob ×

Charly ×

× ▼

☐ Rolling average

Chart Dataframe



Streamlit for data analysis

Reproducibility, interactivity, and communication

- Streamlit as the new IDE?
 - Code must run -> reproducibility
 - Interactive widgets -> flexibility
 - Richer layout -> visual communication
 - Multi-page structure -> clear roadmap
- Examples of my projects
 - Data analysis from EMP / M2P
 - Research paper draft
 - Theory visualization

Project ideas

Project requirements

Hobby, research, or business

- Preliminary plan and data - pitches on Week 6
 - Must be interesting! Hobby, academic, startup or combined
- Define your own challenges, rubric, and use of AI
- Technical requirements:
 - Streamlit app, Jupyter Notebook or Python scripts
 - Fundamental skills: base Python + data cleaning + analysis + visualisation + Git
 - Bonus: stat modelling, automated pipeline, live updates (use of APIs), web scraping, unstructured data, use of LLMs, etc.

Homework

Homework (~5 hours)

See [week3_homework.pdf](#)

- Python fundamentals (~2 hr)
 - User-defined functions
 - New packages: numpy, pandas, streamlit
- Read “Tidy Data” by Hadley Wickham (~1 hr)
- Brainstorm about project ideas (~1 hr)
- Practice data cleaning with *pandas* (~1 hr)
 - Disney movies dataset or any dataset of your choice